

PPO

Proximal Policy Optimization (PPO) Algorithm Study Notes

Author: 千喜Qx

Date: 2026/06/07

Github Link: https://github.com/X2024-AI/reinforcement-learning-notes_Qx

Table of Contents

- [Chapter 1: Background and Motivation](#)
 - [From Policy Gradient to PPO](#)
 - [PPO's Design Goal](#)
- [Chapter 2: Standard Policy Gradient Foundation](#)
 - [Objective Function](#)
 - [Policy Gradient Theorem](#)
 - [Pseudo-Objective Function](#)
- [Chapter 3: Off-Policy via Importance Sampling](#)
 - [The Need for Off-Policy Learning](#)
 - [Importance Sampling Identity](#)
 - [The Probability Ratio](#)
 - [Conservative Policy Iteration \(CPI\) Objective](#)
- [Chapter 4: The Danger of Destructive Updates](#)
 - [Why Unconstrained \$L_{CPI}\$ is Dangerous](#)
 - [The First-Order Approximation Trap](#)
 - [TRPO's Solution: Hard KL Constraint](#)
 - [The Fisher Information Matrix Nightmare](#)
 - [PPO's Ingenious Solution](#)
- [Chapter 5: PPO Clipped Surrogate Objective](#)
 - [The Clipping Interval](#)
 - [Definition of the Clipped Surrogate Objective](#)
 - [Case Analysis: How the Clip Mechanics Work](#)
 - [The Emergency Brake Mechanism](#)
 - [Mathematical Summary: How Clipping Forces Gradient to Zero](#)

- [Chapter 6: The Full PPO Objective Function](#)
 - [Complete Objective with Value Function and Entropy Bonuses](#)
 - [Component 1: Clipped Surrogate Objective \$L_{CLIP}\$](#)
 - [Component 2: Value Function Loss \$L^{VF}\$](#)
 - [Component 3: Entropy Bonus \$S\[\pi_\theta\]\$](#)
- [Chapter 7: Advantage Estimation](#)
 - [\[Mni+16\] Style: Truncated Horizon](#)
 - [The Truncated Estimator \(Equation 10\)](#)
 - [Generalized Advantage Estimation \(GAE\)](#)
 - [Deriving GAE from First Principles](#)
 - [Bias-Variance Tradeoff via \$\lambda\$](#)
- [Chapter 8: Understanding the PPO Objective Curve](#)
 - [Interpolation Along the Policy Update Direction](#)
 - [Reading the Plot: Curve by Curve](#)
 - [Why Does the Red Line Drop?](#)
 - [The Ultimate Takeaway](#)
- [Chapter 9: On-Policy vs Off-Policy in PPO](#)
 - [Clarifying the Nomenclature](#)
 - [The Mathematical Expectation Constraint](#)
 - [Why Standard PG Fails on Reused Data](#)
 - [How PPO Overcomes This \(Importance Sampling\)](#)
 - [Addressing Your Point About Inaccurate Advantage Functions](#)
- [Chapter 10: PPO Algorithm Procedure](#)
 - [Decoding the Optimization Instruction](#)
 - [The Actual Step-by-Step Procedure](#)
 - [The Calculus of How Clipping Restricts Updates](#)
- [Chapter 11: Complete PPO Implementation](#)
 - [PyTorch Implementation with Detailed Comments](#)
 - [Code Key Points Explained](#)
 - [Hyperparameter Meanings](#)
- [Chapter 12: Practical Projects](#)
 - [Project 1: LunarLander-v3 \(Discrete Control\)](#)
 - [Project 2: BipedalWalker-v3 \(Continuous Control Transition\)](#)
 - [Project 3: Atari Pong \(Vision-based RL\)](#)
 - [Project 4: Custom Trading Environment](#)
- [Appendix A: Mathematical Tools](#)
 - [Stochastic Gradient Descent \(SGD\)](#)

- [nn.Tanh\(\) Activation Function](#)
 - [Appendix B: Symbol Table](#)
 - [作者信息](#)
-

Chapter 1: Background and Motivation

From Policy Gradient to PPO

Proximal Policy Optimization (PPO) represents a significant advancement in reinforcement learning algorithms. To understand its motivation, we must first examine the landscape of existing policy optimization methods.

Traditional Policy Gradient methods, such as REINFORCE, are elegant in their simplicity but suffer from a critical flaw: they are **on-policy** algorithms, meaning we must discard all collected data every time we update the policy parameters. This leads to extremely low sample efficiency.

At the other extreme, Trust Region Policy Optimization (TRPO) introduced a mathematically rigorous approach to constraining policy updates using KL divergence, ensuring that the new policy does not deviate too far from the old one. However, TRPO requires computing the **Fisher Information Matrix** (the second derivative of KL divergence), which is computationally prohibitive for large neural networks.

PPO strikes a balance: it retains the mathematical safety of TRPO's constrained optimization while achieving the computational simplicity of standard Policy Gradient methods.

PPO's Design Goal

The core innovation of PPO is the **clipped surrogate objective function**. Instead of using a hard KL constraint like TRPO, PPO modifies the objective function itself to penalize policy updates that deviate too far from the old policy. This clipping mechanism is computationally trivial to implement while effectively preventing destructive policy updates.

Chapter 2: Standard Policy Gradient Foundation

Objective Function

In reinforcement learning, our fundamental goal is to maximize the expected cumulative reward under a policy π_θ . The expected return is defined as:

$$J(\theta) = E_{\tau \sim \pi_\theta}[R(\tau)]$$

where:

- $\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots)$ denotes a trajectory (sequence of states and actions)
- $R(\tau) = \sum_{t=0}^T r_t$ is the total reward of the trajectory
- π_θ is the policy parameterized by neural network weights θ

Policy Gradient Theorem

Using the likelihood ratio trick, we can derive the gradient of this objective without computing the gradient of the environment dynamics:

$$\nabla_\theta J(\theta) = E_{\tau \sim \pi_\theta} \left[\left(\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \right) R(\tau) \right]$$

Through careful application of the **causality principle** (rewards before time t do not affect the gradient at time t), this simplifies to:

$$\nabla_\theta J(\theta) = \hat{E}_t \left[\nabla_\theta \log \pi_\theta(a_t | s_t) \hat{A}_t \right]$$

where $\hat{A}_t$ is the estimated **advantage function** at time step t .

Symbol definitions:

- $\hat{E}_t[\cdot]$: Empirical average over time steps (approximately the expectation)
- $\nabla_\theta \log \pi_\theta(a_t | s_t)$: Gradient of the log probability of taking action a_t in state s_t
- $\hat{A}_t$: Advantage estimate, measuring how much better an action is compared to the average behavior at that state

Pseudo-Objective Function

The corresponding pseudo-objective function (which we maximize via gradient ascent) is:

$$L_{PG}(\theta) = \hat{E}_t \left[\log \pi_\theta(a_t | s_t) \hat{A}_t \right]$$

This is the foundation upon which PPO builds.

Chapter 3: Off-Policy via Importance Sampling

The Need for Off-Policy Learning

Standard policy gradient methods are **on-policy**: we must throw away collected data and sample new trajectories every single time θ is updated. This is extremely sample-inefficient.

To improve data efficiency, we want to optimize a new policy π_θ using data collected by an older policy $\pi_{\theta_{old}}$.

Importance Sampling Identity

We achieve this using **Importance Sampling**, a mathematical identity that rewrites an expectation under one distribution as an expectation under another:

$$E_{x \sim p}[f(x)] = \int p(x) f(x) dx = \int q(x) \frac{p(x)}{q(x)} f(x) dx = E_{x \sim q} \left[\frac{p(x)}{q(x)} f(x) \right]$$

Key insight: The expectation under distribution p can be computed using samples from distribution q , with an importance weight $\frac{p(x)}{q(x)}$ to correct for the distribution mismatch.

The Probability Ratio

Applying this to our policy objective, we define the **probability ratio** $r_t(\theta)$:

$$r_t(\theta) = \frac{\pi_{\theta_{old}}(a_t | s_t)}{\pi_\theta(a_t | s_t)}$$

Important observation: If $\theta = \theta_{old}$, then $r_t(\theta) = 1$.

Conservative Policy Iteration (CPI) Objective

Replacing the standard log-gradient with this ratio yields the Conservative Policy Iteration objective:

$$L_{CPI}(\theta) = \hat{E}_t \left[\frac{\pi_{\theta_{old}}(a_t | s_t)}{\pi_\theta(a_t | s_t)} \hat{A}_t \right] = \hat{E}_t \left[r_t(\theta) \hat{A}_t \right]$$

Symbol definitions:

- $\pi_{\theta_{old}}(a_t | s_t)$: Probability of taking action a_t in state s_t under the **old** (behavior) policy
- $\pi_\theta(a_t | s_t)$: Probability under the **new** (target) policy being optimized
- $r_t(\theta)$: Ratio measuring how much more (or less) likely the new policy is to select a_t compared to the old policy

Chapter 4: The Danger of Destructive Updates

Why Unconstrained L_{CPI} is Dangerous

Maximizing $L_{CPI}(\theta)$ without constraints is highly dangerous. Because L_{CPI} is only a **first-order (linear) approximation** of the true objective around θ_{old} , taking a massive gradient step can

cause $r_t(\theta)$ to skyrocket or plunge. This forces the policy into an unrecoverable state where performance collapses.

The First-Order Approximation Trap

In machine learning, the true objective function (how good your policy actually is across all possible neural network weights) is a highly complex, winding mountain range.

When an algorithm takes a first-order approximation, it:

1. Looks at the mountain at a single exact coordinate (θ_{old})
2. Calculates the slope (the first derivative/gradient)
3. Draws a **perfectly straight tangent line**

Locally (a tiny step): The straight line is highly accurate. Moving 0.001 units along the line, you are still basically on the mountain.

Globally (a massive step): The straight line completely loses touch with reality. If the mountain suddenly turns into a steep downward cliff, the straight line doesn't care—it keeps pointing straight up into the sky.

For actions that were good ($\hat{A}_t > 0$), the linear approximation tells the network: "The further you move in this direction, the higher your reward will go, infinitely!" The optimizer blindly trusts this line and takes a massive step, forcing $r_t(\theta)$ to infinity.

Conversely, for bad actions ($\hat{A}_t < 0$), the line tells it to make the probability ratio plunge to zero, completely breaking the policy distribution.

TRPO's Solution: Hard KL Constraint

TRPO recognized this danger and introduced a **hard mathematical constraint** using KL Divergence to ensure π_θ stays close to $\pi_{\theta_{old}}$:

$$\theta_{max} \quad \hat{E}_t \left[r_t(\theta) \hat{A}_t \right] \quad \text{subject to} \quad \hat{E}_t [KL(\pi_{\theta_{old}}(\cdot|s_t) || \pi_\theta(\cdot|s_t))] \leq \delta$$

Symbol definitions:

- $KL(\pi_{\theta_{old}} || \pi_\theta)$: KL divergence measuring how much the policy distribution changes
- δ : Maximum allowed KL divergence (the "trust region" radius)

The Fisher Information Matrix Nightmare

Why TRPO is computationally grueling:

The shape of the "trust region" isn't a perfect circle—it's a highly distorted, multi-dimensional ellipsoid. In some weight directions, a tiny change alters the policy completely; in other directions, you can change the weights a lot without altering behavior.

To map out the exact contours of this safe region, TRPO must compute how the gradient of the KL divergence changes as the weights change. The derivative of a gradient is a **second derivative** (a matrix of partial second derivatives called a **Hessian**).

When applied to probability distributions, this specific matrix of second derivatives is called the **Fisher Information Matrix (FIM)**.

The computational scale:

Suppose your neural network contains $P = 1,000,000$ parameters (weights and biases). The FIM is a square matrix of size $P \times P$, containing 1 trillion elements. Storing this requires approximately **4 Terabytes of VRAM**.

To make an update step, TRPO must compute the inverse of this matrix (F^{-1}), which is computationally devastating.

PPO's Ingenious Solution

PPO looked at this computational nightmare and said: "Instead of spending massive computational power building a perfect 4-Terabyte safe-room matrix (TRPO), let's use our cheap, linear approximation but add a **simple clip function** to cut off the gradient if it tries to step too far."

Chapter 5: PPO Clipped Surrogate Objective

The Clipping Interval

Instead of forcing a hard constraint on the policy space, PPO forces the probability ratio $r_t(\theta)$ to stay within an interval around 1, typically:

$$[1 - \epsilon, 1 + \epsilon]$$

where $\epsilon \approx 0.2$ is a hyperparameter.

Definition of the Clipped Surrogate Objective

The **PPO Clipped Surrogate Objective** is defined as:

$$L_{CLIP}(\theta) = \hat{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$

Symbol definitions:

- $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)$: Clips $r_t(\theta)$ to the interval $[1 - \epsilon, 1 + \epsilon]$
- $\min(\cdot, \cdot)$: Takes the minimum of the unclipped and clipped objectives

Case Analysis: How the Clip Mechanics Work

To understand why the $\min$ and clip operators work, we must examine their behavior depending on whether the action yielded a positive or negative outcome.

Case 1: Positive Advantage ($\hat{A}_t > 0$)

The action taken resulted in a better-than-expected return. We want to modify θ to make this action more likely, which drives $r_t(\theta)$ **above 1**.

Sub-case 1a: When $r_t(\theta) \leq 1 + \epsilon$

The clipping function does not trigger. The objective is simply:

$$L_{CLIP}(\theta) = r_t(\theta)\hat{A}_t$$

The gradient is active, and the network learns to increase the probability of this good action.

Sub-case 1b: When $r_t(\theta) > 1 + \epsilon$

The clipped term locks at $(1 + \epsilon)\hat{A}_t$. Because $\hat{A}_t > 0$:

$$r_t(\theta)\hat{A}_t > (1 + \epsilon)\hat{A}_t$$

The $\min(\cdot)$ operator selects the **smaller** value: $(1 + \epsilon)\hat{A}_t$.

Intuition: Once the policy has made the action $1 + \epsilon$ times more likely than before, the gradient **drops to zero**. This prevents the algorithm from over-optimizing on a single good trajectory.

Case 2: Negative Advantage ($\hat{A}_t < 0$)

The action taken resulted in a worse-than-expected return. We want to modify θ to make this action less likely, which drives $r_t(\theta)$ **below 1**.

Sub-case 2a: When $r_t(\theta) \geq 1 - \epsilon$

The clipping function does not trigger. The objective is:

$$L_{CLIP}(\theta) = r_t(\theta)\hat{A}_t$$

Sub-case 2b: When $r_t(\theta) < 1 - \epsilon$

The clipped term locks at $(1 - \epsilon)\hat{A}_t$. Because $\hat{A}_t < 0$, multiplying it by a smaller number yields a **larger (less negative)** value:

$$0.5 \times (-2) = -1.0 \quad \text{while} \quad 0.8 \times (-2) = -1.6$$

The $\min(-1.0, -1.6)$ operator selects the **smaller (more negative)** value: $(1 - \epsilon)\hat{A}_t$.

Intuition: The objective flattens out when $r_t(\theta)$ goes below $1 - \epsilon$, meaning the agent isn't given infinite praise for making a bad action even less likely.

The Emergency Brake Mechanism

What if the policy makes a terrible mistake?

If the action was bad ($\hat{A}_t < 0$) but the policy update accidentally made it way more likely ($r_t(\theta) \gg 1$):

- The clipped term becomes $(1 + \epsilon)\hat{A}_t$ (a large negative number)
- The unclipped term $r_t(\theta)\hat{A}_t$ becomes a **massive negative number**
- The $\min(\cdot)$ operator selects the unclipped value

This acts as an **unclipped emergency brake**, allowing a powerful negative gradient to aggressively drag the policy back.

Mathematical Summary: How Clipping Forces Gradient to Zero

To understand exactly how the clip restricts the update, we examine what happens to the gradient during backpropagation.

Recall the core optimization update step:

$$\theta_{new} = \theta_{old} + \eta \nabla_{\theta} L_{CLIP}(\theta)$$

If $\nabla_{\theta} L_{CLIP}(\theta) = 0$, then $\theta_{new} = \theta_{old}$, meaning the network **completely stops learning** from that data point.

When $r_t(\theta) > 1 + \epsilon$:

$$L_{CLIP}(\theta) = (1 + \epsilon)\hat{A}_t$$

Look closely at this term. ϵ is a constant (like 0.2) and $\hat{A}_t$ was already calculated and frozen during data collection. **There is no θ left in this term.** It is a flat, constant number.

Therefore:

$$\nabla_{\theta} L_{CLIP}(\theta) = \nabla_{\theta} [\text{constant}] = 0$$

Summary: The clipping mechanism doesn't physically block the weights; it **flattens the loss landscape**. By turning the objective function into a flat plateau whenever the policy deviates too far, it strips the optimizer of its gradient. Without a gradient, the optimizer cannot calculate a direction to move, effectively locking the policy in place.

Chapter 6: The Full PPO Objective Function

Complete Objective with Value Function and Entropy Bonuses

When implementing PPO using deep neural networks, the policy network (Actor) and value network (Critic) often share early layers. To optimize both simultaneously while preventing premature convergence, the final PPO loss function combines three components:

$$L^{CLIP+VF+S}(\theta) = \hat{E}_t [L_{CLIP}^t(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)]$$

where:

$$L_t^{VF}(\theta) = (V_\theta(s_t) - V_{targ}^t)^2$$

Symbol definitions:

- $L_{CLIP}^t(\theta)$: Clipped surrogate objective at time t
- $L_t^{VF}(\theta)$: **Value function loss** - squared error between predicted value $V_\theta(s_t)$ and target value V_{targ}^t
- $S[\pi_\theta](s_t)$: **Entropy bonus** measuring the randomness of the policy at state s_t
- c_1 : Coefficient for value function loss (typically 0.5)
- c_2 : Coefficient for entropy bonus (typically 0.01)

Component1: Clipped Surrogate Objective L_{CLIP}

As derived in Chapter 5, this is the core PPO objective that prevents destructive policy updates:

$$L_{CLIP}(\theta) = \hat{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$

Component 2: Value Function Loss L^{VF}

The value function loss forces the critic to accurately predict state values:

$$L_t^{VF}(\theta) = (V_\theta(s_t) - V_{targ}^t)^2 = (V_\theta(s_t) - \hat{R}_t)^2$$

Why this matters:

- The critic network $V_\theta(s)$ estimates the expected cumulative reward from state s
- V_{target}^t is the target value (often computed using GAE or n-step returns)
- Minimizing this MSE loss improves the accuracy of advantage estimates
- Better advantage estimates lead to more stable and efficient learning

Component 3: Entropy Bonus $S[\pi_\theta]$

Entropy measures the **randomness** or **uncertainty** of a policy distribution:

$$S[\pi_\theta](s) = - \sum_a \pi_\theta(a|s) \log \pi_\theta(a|s)$$

For a discrete action space with softmax policy, this is the Shannon entropy of the action probability distribution.

The role of entropy bonus:

- **Encourages exploration:** A deterministic policy (all probability on one action) has zero entropy. Adding an entropy bonus prevents the policy from collapsing too early
- **Prevents premature convergence:** Without entropy bonus, the policy might lock into a suboptimal deterministic policy too early
- **The coefficient c_2 :** Controls the strength of exploration. Too high = random behavior; too low = premature convergence

Important note: Entropy bonus is subtracted in the loss function, meaning we want to **maximize** entropy while maximizing the clipped objective. This is achieved by adding $-c_2 \times S$ (negative entropy) to the loss, which is equivalent to minimizing $c_2 \times (-S)$.

Chapter 7: Advantage Estimation

[Mni+16] Style: Truncated Horizon

One style of policy gradient implementation, popularized in [Mni+16] (the Mnih et al. 2016 A3C paper) and well-suited for use with **recurrent neural networks**, runs the policy for T timesteps (where T is much less than the episode length), and uses the collected samples for an update.

This style requires an advantage estimator that does not look beyond timestep T .

The Truncated Estimator (Equation 10)

The estimator used by [Mni+16] is:

$$\hat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V(s_T)$$

where t specifies the time index in $[0, T]$, within a given length- T trajectory segment.

Symbol definitions:

- $V(s_t)$: Value estimate at the start of the truncated segment
- $r_t, r_{t+1}, \dots, r_{T-1}$: Observed rewards within the truncated window
- $\gamma^{T-t}V(s_T)$: Bootstrap value at the end of the truncated segment
- The negative sign before $V(s_t)$ corrects for the starting value (baseline subtraction)

Intuition: This estimator looks exactly $T - t$ steps into the future (plus a bootstrap value at the end), making it suitable for truncated BPTT (backpropagation through time) in RNNs.

Generalized Advantage Estimation (GAE)

Generalizing the truncated choice, we can use a **truncated version of generalized advantage estimation**, which reduces to Equation (10) when $\lambda = 1$:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$$

where the **TD error** δ_t is defined as:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Symbol definitions:

- δ_t : TD error at time t (how much the value estimate was off by)
- γ : Discount factor
- λ : GAE smoothing parameter (balancing bias and variance)
- $V(s_{t+1})$: Value estimate at the next state

Deriving GAE from First Principles

The GAE formula represents an exponentially weighted moving average of n -step TD errors:

$$\hat{A}_t = \sum_{n=0}^{T-t-1} (\gamma\lambda)^n \delta_{t+n}$$

Let's verify this reduces to Equation (10) when $\lambda = 1$:

$$\hat{A}_t = \delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2} + \dots + \gamma^{T-t-1}\delta_{T-1}$$

Expanding each δ :

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$$\delta_{t+1} = r_{t+1} + \gamma V(s_{t+2}) - V(s_{t+1})$$

⋮

After algebraic cancellation (telescoping sum), this simplifies to exactly Equation (10).

Bias-Variance Tradeoff via λ

GAE_LAMBDA (λ) is the tuning knob for balancing **two evils**:

Extreme 1: $\lambda = 0$ (High Bias, Low Variance)

- The algorithm only looks one step into the future
- Relies heavily on the Critic network's value estimates
- Early in training, when the Critic is clueless, this leads to systematic bias

Extreme 2: $\lambda = 1$ (Low Bias, High Variance)

- The algorithm looks all the way to the end of the episode (Monte Carlo returns)
- Doesn't rely on value estimates (unbiased)
- However, stochastic environments cause variance to compound exponentially over long horizons

The Sweet Spot: $\lambda = 0.95$

By setting GAE_LAMBDA = 0.95, we take an exponentially weighted average of all multi-step look-aheads:

- Leans heavily toward looking far into the future (low bias)
- Applies a gentle discount factor to dampen chaotic noise from later steps (controlled variance)

Decades of empirical RL research have shown that **0.95** is almost universally the optimal choice for this parameter.

Chapter 8: Understanding the PPO Objective Curve

Interpolation Along the Policy Update Direction

Neural network weights live in a space with thousands or millions of dimensions, making it impossible to visualize the entire loss landscape at once. To solve this, researchers take a single **1D "slice"** through that massive space.

Let θ_{old} be the starting weights and θ_{new} be the target weights proposed by PPO. The "direction" is the straight line connecting them:

$$\theta(\alpha) = \theta_{old} + \alpha(\theta_{new} - \theta_{old})$$

Reading the x-axis (Linear interpolation factor α):

- At $\alpha = 0$: Standing exactly at θ_{old} (before making any update)
- At $\alpha = 1$: Standing exactly at θ_{new} (the actual step size PPO decided to take)
- At $\alpha > 1$: Intentionally overshooting to see what would happen

Reading the Plot: Curve by Curve

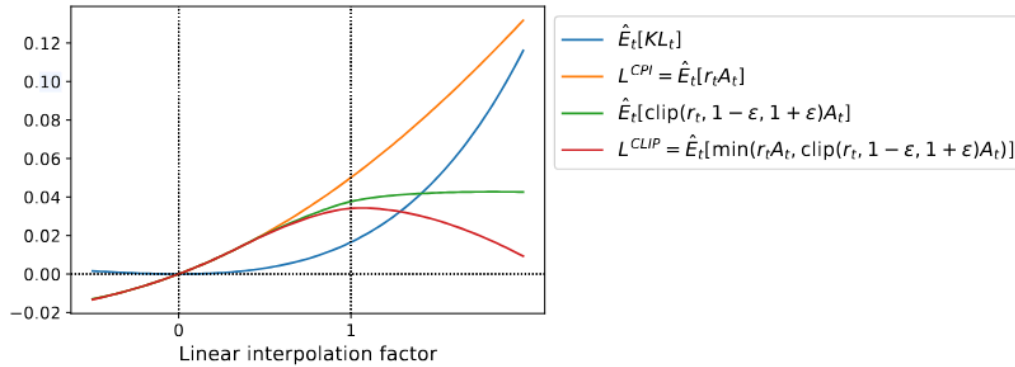


Figure 2: Surrogate objectives, as we interpolate between the initial policy parameter θ_{old} , and the updated policy parameter, which we compute after one iteration of PPO. The updated policy has a KL divergence of about 0.02 from the initial policy, and this is the point at which L^{CLIP} is maximal. This plot corresponds to the first policy update on the **Hopper-v1 problem**, using hyperparameters provided in Section 6.1.

The plot shows how four different mathematical metrics change as we dial α up or down.

The Orange Line (L_{CPI}): The Dangerous Optimist

This is the standard, unclipped policy gradient objective ($r_t \hat{A}_t$). Notice that as we push past $\alpha = 1$, the orange line keeps skyrocketing upward.

The Problem: If you used standard policy gradient, the optimizer would say "Wow, going right yields massive rewards! Let's take a giant leap to $\alpha = 5$ or $\alpha = 10$!" This leads to destructive policy updates.

The Blue Line ($\hat{E}_t[KL_t]$): The Hidden Cliff

This tracks the KL Divergence (how much the policy's actual behavior changes). Notice how it is flat at $\alpha = 0$, but curves **violently upward** as α increases.

The Problem: This shows the danger the orange line is blind to. If the optimizer blindly follows the orange line to the right, the policy changes so drastically that performance will utterly collapse.

The Green Line (Clipped Term Only)

This shows the behavior of just the isolated clipping piece: $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)\hat{A}_t$. As you move to the right, it completely **flattens out**, stripping the optimizer of any gradient once it gets too far.

The Red Line (L_{CLIP}): PPO's Masterstroke

This is the actual PPO objective function, which takes the **minimum** of the orange and green lines. Notice its beautiful shape:

- Between 0 and 1, it climbs upward, allowing the model to learn efficiently
- It peaks almost exactly at $\alpha = 1$
- As soon as you overshoot past 1, the red line starts dropping **rapidly**

Why Does the Red Line Drop? (The "Lower Bound" Intuition)

PPO's L_{CLIP} is a **lower bound** on L_{CPI} , with a penalty for having too large of a policy update.

Why does it drop instead of just staying flat?

An iteration of PPO calculates the average (expectation $\hat{E}_t$) over an **entire batch** of data containing hundreds of different state-action pairs.

For actions that were good ($\hat{A}_t > 0$), overshooting makes them flat (following the green line).

But for actions that were bad ($\hat{A}_t < 0$), taking too large of a step might accidentally make those bad actions **way more likely**. For those specific bad transitions, the unclipped term ($r_t \hat{A}_t$) becomes a **massive negative number**.

Because the PPO formula uses a $\min(\cdot)$ operator, it forces the objective to ignore the flat green lines of the good actions and instead focus on the **massive negative penalties** of the bad actions. This drags the entire average (the red line) downward.

The Ultimate Takeaway

PPO's math effectively reshapes the loss landscape. It turns a dangerous, blindingly steep hill (the orange line) into a safe, well-defined peak (the red line) that forces your deep learning optimizer to **stop exactly at the perfect step size** ($\alpha = 1$) where the update is safe.

Chapter 9: On-Policy vs Off-Policy in PPO

Clarifying the Nomenclature

Strictly speaking, PPO is an on-policy algorithm. In RL, an algorithm is off-policy (like DQN or SAC) if it can store experiences in a replay buffer and reuse data collected hundreds of thousands of steps ago by ancient versions of the policy. PPO cannot do this. If you try to feed PPO data from an entirely different training session, the algorithm will fail.

However, PPO is often described as having **off-policy characteristics** because it is **locally off-policy**. It collects a batch of data using a policy, freezes that policy, and then reuses that exact same data to update its weights for **K epochs**.

The Mathematical Expectation Constraint

Your intuition about "inaccuracy after an update" is very close to the truth, but it's not about the accuracy of the total reward (G_t) or the Advantage ($\hat{A}_t$). It is about a **fundamental rule of probability**.

Look closely at the standard Policy Gradient objective function:

$$L_{PG}(\theta) = E_{(s_t, a_t) \sim \pi_\theta} \left[\log \pi_\theta(a_t | s_t) \hat{A}_t \right]$$

Pay attention to the subscript under the expectation: $(s_t, a_t) \sim \pi_\theta$. This means this equation is **mathematically valid only if** the states and actions in your data pool were sampled directly from the **exact policy** π_θ you are currently optimizing.

Why Standard PG Fails on Reused Data

Here is the breakdown:

1. You collect data using your current policy weights θ_0
2. You run one step of gradient ascent. Your weights change from $\theta_0 \rightarrow \theta_1$
3. You now have a new policy, π_{θ_1}
4. If you try to take a second gradient step using that same data, you are evaluating $L_{PG}(\theta_1)$ using data that came from π_{θ_0}
5. Because $\pi_{\theta_1} \neq \pi_{\theta_0}$, the **mathematical expectation collapses**. The gradient points in a fundamentally wrong direction, causing the policy to destructively overfit to a distribution it no longer represents.

How PPO Overcomes This (Importance Sampling)

PPO bypasses this expectation constraint using **Importance Sampling**. This is a mathematical identity that allows you to calculate an expectation under distribution P using data sampled from distribution Q .

Instead of optimizing the standard log-loss, PPO optimizes a ratio:

$$r_t(\theta) = \frac{\pi_{\theta_{old}}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)}$$

The objective function becomes:

$$L_{CPI}(\theta) = E_{(s_t, a_t) \sim \pi_{\theta_{old}}} \left[\frac{\pi_{\theta_{old}}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)} \hat{A}_t \right]$$

Notice the subscript now: the data is allowed to come from $\pi_{\theta_{old}}$.

When PPO runs for K epochs, $\pi_{\theta_{old}}$ stays completely frozen. As θ updates and changes over those K epochs, the ratio $r_t(\theta)$ **mathematically corrects for the fact that the data is slightly stale**. It scales the gradient up or down to compensate for the distribution shift.

Addressing Your Point About Inaccurate Advantage Functions

There is a **huge difference** between:

1. **Statistical noise** (an inaccurate Advantage function): The Critic network is new and makes poor predictions. However, in deep learning, neural networks are excellent at filtering out zero-mean random noise if you feed them enough data. As long as the gradients point in the generally correct direction on average, the model will learn.
2. **Systematic mathematical error** (violating the expectation constraint): If you reuse data in standard Policy Gradient without Importance Sampling, you aren't just dealing with noisy data—you are feeding the optimizer a **mathematically fraudulent gradient**. The optimizer thinks it is calculating the slope of the current policy's landscape, but it is actually calculating a distorted slope from an old policy's landscape. This systematic bias rapidly drives the policy network off a cliff.

Chapter 10: PPO Algorithm Procedure

Decoding the Optimization Instruction

The phrase "**Optimize surrogate L wrt θ , with K epochs and minibatch size $M \leq NT$** " is just compact machine learning shorthand for a data-recycling pipeline.

Variable	What it is	What it actually means
N	Parallel Environments	The number of simultaneous game/simulation instances running at the same time to collect data
T	Timesteps	How many steps of experience each environment collects before pausing to update the network

Variable	What it is	What it actually means
NT	The Total Data Pool	The complete batch of experience collected ($N \times T$ total transitions)
M	Minibatch Size	The smaller chunk of data sampled from NT used to calculate a single gradient step
K	Epochs	How many times the algorithm loops through the entire pool of NT data before throwing it away

The Actual Step-by-Step Procedure

Instead of collecting data, updating once, and immediately throwing it away (like standard Policy Gradient), PPO does the following:

Step 1: Collect Data

- The agent plays in N environments for T steps
- It accumulates a big pool of NT total data points: states, actions, rewards, values, log probabilities, done flags

Step 2: Freeze Old Policy

- Save the current policy weights as $\pi_{\theta_{old}}$
- This is used to compute the denominator of the ratio $r_t(\theta)$

Step 3: The Epoch Loop (Runs K times)

For each epoch:

1. Shuffle the entire pool of NT data points
2. Chop that data pool into smaller minibatches of size M
3. For each minibatch:
 - Calculate the PPO loss (L_{CLIP})
 - Compute gradients and update the actual network weights θ using an optimizer (Adam or SGD)

Step 4: Discard and Repeat

- After K full passes over the data, throw the NT data away
- Repeat from Step 1 with the updated policy

The Calculus of How Clipping Restricts Updates

To understand exactly how the clip restricts the update, we examine what happens to the gradient during backpropagation.

Recall the core optimization update step:

$$\theta_{new} = \theta_{old} + \eta \nabla_{\theta} L_{CLIP}(\theta)$$

If $\nabla_{\theta} L_{CLIP}(\theta) = 0$, then $\theta_{new} = \theta_{old}$, meaning the network **completely stops learning** from that data point.

Zone 1: The Normal Zone (No Clipping)

When the new policy hasn't changed much from the old policy, the ratio is close to 1 ($1 - \epsilon < r_t(\theta) < 1 + \epsilon$). The `min` operator selects the unclipped objective:

$$L_{CLIP}(\theta) = r_t(\theta) \hat{A}_t = \frac{\pi_{\theta_{old}}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)} \hat{A}_t$$

When we take the derivative with respect to the network weights θ :

$$\nabla_{\theta} L_{CLIP}(\theta) = \frac{\pi_{\theta_{old}}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)} \cdot \frac{-\nabla_{\theta} \pi_{\theta}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)} \cdot \hat{A}_t = -\frac{\pi_{\theta_{old}}}{\pi_{\theta}^2} \nabla_{\theta} \pi_{\theta} \hat{A}_t$$

This gradient is **active** and drives weight updates.

Zone 2: The Clipped Zone (The Update "Brake")

Now imagine the network updates its weights, and in the next epoch, it encounters the exact same data point. The policy has changed so much that it is now highly likely to pick that action. The ratio shoots up past the threshold: $r_t(\theta) > 1 + \epsilon$.

At this point, the `min` operator triggers and forces the objective function to choose the clipped version:

$$L_{CLIP}(\theta) = (1 + \epsilon) \hat{A}_t$$

Look closely at this term. ϵ is a constant (like 0.2) and $\hat{A}_t$ was already calculated and frozen during data collection. **There is no θ left in this term.** It is a flat, constant number.

Now, look at what happens when we try to compute the gradient:

$$\nabla_{\theta} L_{CLIP}(\theta) = \nabla_{\theta} [\text{constant}] = 0$$

Because the gradient drops to **exactly zero**, the SGD update formula evaluates to:

$$\theta \leftarrow \theta + \eta(0) = \theta$$

The network **stops learning** from this data point.

Chapter 11: Complete PPO Implementation

PyTorch Implementation with Detailed Comments

```
import gymnasium as gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions.categorical import Categorical

# Hyperparameters
ENV_NAME = "CartPole-v1"
LEARNING_RATE = 3e-4
TOTAL_TIMESTEPS = 100000
TRAIN_COEFFICIENTS = {"value": 0.5, "entropy": 0.01}

# PPO Specific Hyperparameters
ROLLOUT_STEPS = 2048      # N * T (Total data pool size per iteration)
MINIBATCH_SIZE = 64      # M
PPO_EPOCHS = 10          # K
CLIP_EPSILON = 0.2        # epsilon
GAMMA = 0.99              # Discount factor
GAE_LAMBDA = 0.95         # Smoothing factor for GAE

class ActorNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super(ActorNetwork, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(state_dim, 64),
            nn.Tanh(),
            nn.Linear(64, 64),
            nn.Tanh(),
            nn.Linear(64, action_dim)
        )

    def get_action_distribution(self, state):
        logits = self.network(state)
        return Categorical(logits=logits)

class CriticNetwork(nn.Module):
    def __init__(self, state_dim):
```

```

    super(CriticNetwork, self).__init__()
    self.network = nn.Sequential(
        nn.Linear(state_dim, 64),
        nn.Tanh(),
        nn.Linear(64, 64),
        nn.Tanh(),
        nn.Linear(64, 1)
    )

    def forward(self, state):
        return self.network(state)

class PPOAgent:
    def __init__(self, state_dim, action_dim):
        self.actor = ActorNetwork(state_dim, action_dim)
        self.critic = CriticNetwork(state_dim)

        self.actor_optimizer = optim.Adam(self.actor.parameters(),
lr=LEARNING_RATE)
        self.critic_optimizer = optim.Adam(self.critic.parameters(),
lr=LEARNING_RATE)

    def train_step(self, states, actions, old_log_probs, returns, advantages):
        # Convert data to PyTorch tensors
        states = torch.tensor(np.array(states), dtype=torch.float32)
        actions = torch.tensor(np.array(actions), dtype=torch.long)
        old_log_probs = torch.tensor(np.array(old_log_probs),
dtype=torch.float32)
        returns = torch.tensor(np.array(returns),
dtype=torch.float32).unsqueeze(1)
        advantages = torch.tensor(np.array(advantages), dtype=torch.float32)

        # Normalize advantages for training stability
        advantages = (advantages - advantages.mean()) / (advantages.std() +
1e-8)

        dataset_size = len(states)

        # K Epochs Loop
        for _ in range(PPO_EPOCHS):
            permutation = torch.randperm(dataset_size)

            # Minibatch Loop
            for start_idx in range(0, dataset_size, MINIBATCH_SIZE):
                batch_indices = permutation[start_idx:start_idx +

```

MINIBATCH_SIZE]

```
        b_states = states[batch_indices]
        b_actions = actions[batch_indices]
        b_old_log_probs = old_log_probs[batch_indices]
        b_returns = returns[batch_indices]
        b_advantages = advantages[batch_indices]

        # Actor Loss Calculation
        dist = self.actor.get_action_distribution(b_states)
        new_log_probs = dist.log_prob(b_actions)
        entropy = dist.entropy().mean()

        # Probability Ratio r_t(theta)
        ratios = torch.exp(new_log_probs - b_old_log_probs)

        # Clipped Surrogate Objective
        surr1 = ratios * b_advantages
        surr2 = torch.clamp(ratios, 1.0 - CLIP_EPSILON, 1.0 +
CLIP_EPSILON) * b_advantages
        actor_loss = -torch.min(surr1, surr2).mean()

        # Critic Loss Calculation (MSE)
        state_values = self.critic(b_states)
        critic_loss = nn.MSELoss()(state_values, b_returns)

        # Total Objective Optimization
        self.actor_optimizer.zero_grad()
        actor_total_loss = actor_loss - TRAIN_COEFFICIENTS["entropy"]
* entropy

        actor_total_loss.backward()
        self.actor_optimizer.step()

        self.critic_optimizer.zero_grad()
        critic_total_loss = TRAIN_COEFFICIENTS["value"] * critic_loss
        critic_total_loss.backward()
        self.critic_optimizer.step()

def main():
    env = gym.make(ENV_NAME)
    state_dim = env.observation_space.shape[0]
    action_dim = env.action_space.n

    agent = PPOAgent(state_dim, action_dim)
```

```

state, _ = env.reset()
global_step = 0
episode_rewards = []
current_episode_reward = 0

while global_step < TOTAL_TIMESTEPS:
    # 1. Storage buffers for Rollout Data Pool (N * T)
    b_states, b_actions, b_log_probs, b_rewards, b_dones, b_values = [],
    [], [], [], [], []

    # 2. Data Collection Phase
    for _ in range(ROLLOUT_STEPS):
        global_step += 1
        state_t = torch.tensor(state, dtype=torch.float32)

        with torch.no_grad():
            dist = agent.actor.get_action_distribution(state_t)
            action = dist.sample().item()
            log_prob = dist.log_prob(torch.tensor(action)).item()
            value = agent.critic(state_t).item()

        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
        current_episode_reward += reward

        b_states.append(state)
        b_actions.append(action)
        b_log_probs.append(log_prob)
        b_rewards.append(reward)
        b_dones.append(done)
        b_values.append(value)

        state = next_state
        if done:
            state, _ = env.reset()
            episode_rewards.append(current_episode_reward)
            if len(episode_rewards) % 10 == 0:
                print(f"Step: {global_step} | Avg Reward (Last 10 Eps):
{np.mean(episode_rewards[-10:]):.2f}")
            current_episode_reward = 0

    # Compute bootstrap value for the final unfinished step
    with torch.no_grad():
        next_value = agent.critic(torch.tensor(state,
dtype=torch.float32)).item() if not done else 0

```

```

# 3. Generalized Advantage Estimation (GAE) Calculation
b_advantages = np.zeros_like(b_rewards, dtype=np.float32)
b_returns = np.zeros_like(b_rewards, dtype=np.float32)
last_gae_lam = 0

for t in reversed(range(ROLLOUT_STEPS)):
    next_non_terminal = 1.0 - b_dones[t]
    next_val = b_values[t + 1] if t < ROLLOUT_STEPS - 1 else
next_value

    # Delta (temporal difference error)
    delta = b_rewards[t] + GAMMA * next_val * next_non_terminal -
b_values[t]

    # GAE tracking formula
    b_advantages[t] = last_gae_lam = delta + GAMMA * GAE_LAMBDA *
next_non_terminal * last_gae_lam
    b_returns[t] = b_advantages[t] + b_values[t]

# 4. Policy Optimization Phase
agent.train_step(b_states, b_actions, b_log_probs, b_returns,
b_advantages)

env.close()

if __name__ == "__main__":
    main()

```

Code Key Points Explained

Code Segment	Mathematical Meaning
<code>Categorical(logits).sample()</code>	Sampling action from softmax policy distribution $\pi_{\theta}(a s)$
<code>torch.clamp(ratios, 1-CLIP_EPSILON, 1+CLIP_EPSILON)</code>	Implementing $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$
<code>torch.min(surr1, surr2)</code>	Implementing the $\min(\cdot, \cdot)$ operator in L_{CLIP}
<code>-torch.min(surr1, surr2).mean()</code>	Negative because we perform gradient descent (minimizing negative PPO loss = maximizing L_{CLIP})
GAE tracking formula	Computing $A_t = \delta_t + \gamma \lambda \delta_{t+1} + \dots$
<code>b_returns[t] = b_advantages[t] + b_values[t]</code>	Computing target values $V_{targ}^t = A_t + V(s_t)$

Hyperparameter Meanings

Hyperparameter	Value	Meaning
CLIP_EPSILON	0.2	Controls how far the policy can deviate from $\pi_{\theta_{old}}$ in one update
GAE_LAMBDA	0.95	Balances bias-variance tradeoff in advantage estimation
ROLLOUT_STEPS	2048	Total samples ($N \times T$) collected before each update
PPO_EPOCHS	10	Number of times we reuse the same data to update θ
MINIBATCH_SIZE	64	Size of each minibatch for gradient computation
GAMMA	0.99	Discount factor for future rewards
LEARNING_RATE	3e-4	Step size for Adam optimizer

Chapter 12: Practical Projects

Project 1: LunarLander-v3 (Discrete Control)

What it is: A classic vector environment where you must safely guide a lunar lander module onto a landing pad using main and side thrusters.

Why it's a great test: It introduces highly unforgiving physics. If you fire the thrusters too early or late, you crash. It serves as an excellent environment for testing your value network's baseline accuracy, as it contains a clear local minimum (hovering forever out of fear of landing).

Project 2: BipedalWalker-v3 (Continuous Control Transition)

What it is: A 2D robot that must learn to walk forward across rough terrain without falling over.

Why it's a great test: This requires modifying your Actor network to handle a **Continuous Action Space** (outputting a Mean μ and Variance σ^2 for a Gaussian distribution instead of discrete logits). PPO shines brightly in continuous robotics tasks; watching your walker go from uncoordinated flailing to a stable sprint is incredibly rewarding.

Project 3: Atari Pong (Vision-based RL)

What it is: Wrapping a classic video game emulator using packages like `gym-super-mario-bros` or `stable-baselines3`'s Atari environments.

Why it's a great test: It forces you to swap out the simple Multi-Layer Perceptrons (MLPs) in the code above for **Convolutional Neural Networks (CNNs)**. Your agent must learn to read raw pixel arrays, extract spatial information, and map it directly to game controls.

Project 4: Custom Trading Environment

What it is: Writing a custom Gymnasium subclass that reads historical stock or cryptocurrency price data. The actions are `[0: Hold, 1: Buy, 2: Sell]`, and the reward is your portfolio's changing net worth.

Why it's a great test: This bridges theory into real-world application. PPO is widely utilized by quantitative trading firms because its clipped objective prevents catastrophic, erratic trading adjustments when financial markets experience sudden shifts in volatility.

Appendix A: Mathematical Tools

Stochastic Gradient Descent (SGD)

The Mathematical Formula:

In standard Gradient Descent, we update our model parameters θ by calculating the gradient of the entire dataset loss function $J(\theta)$.

In Stochastic Gradient Descent, we approximate that true gradient by calculating the loss for just one randomly selected training example (or a single mini-batch) at each iteration.

The mathematical update rule for SGD at step t is:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L_i(\theta_t)$$

Symbol definitions:

- θ_t : Current vector of model parameters at step t
- θ_{t+1} : Updated parameters for the next step
- η (Eta): Learning rate - determines how large of a step we take down the gradient
- i : A randomly selected index from the training dataset
- $L_i(\theta_t)$: Loss function evaluated on training instance (x_i, y_i)
- $\nabla_{\theta} L_i(\theta_t)$: Gradient (vector of partial derivatives) of the loss with respect to θ

The Trade-Offs:

The Good	The Bad
Computationally Cheap: Calculating the gradient for 1 sample requires drastically less memory	Noisy Paths: The parameter updates have high variance
Faster Convergence: Because it updates parameters constantly	Never Truly Settles: Due to the noise
Online Learning: Can update on the fly as new data streams in	

nn.Tanh() Activation Function

What is nn.Tanh()?

nn.Tanh() stands for the **Hyperbolic Tangent** activation function. It squashes any input number into a strict, predictable range between -1.0 and 1.0:

$$\tanh(x) = \frac{e^x + e^{-x}}{e^x - e^{-x}}$$

Why use Tanh instead of ReLU in PPO?

While nn.ReLU() (which turns all negative numbers to 0) is the king of computer vision, Tanh is often preferred in reinforcement learning policies:

- 1. **Zero-Centered Outputs:** It maps negative inputs to negative outputs and positive inputs to positive outputs, centering its behavior around zero. This makes the gradient updates cleaner and more balanced when shifting probabilities up or down.
- 2. **Smoother Distributions:** Tanh is a smooth, continuous curve. Because the Actor network outputs probabilities (logits) to form a policy distribution, smooth curves prevent sudden, jarring spikes in action probabilities.

Appendix B: Symbol Table

Symbol	Meaning
$\pi_{\theta}(a s)$	Parameterized policy - probability of taking action a in state s
θ	Policy network parameters (weights and biases)

Symbol	Meaning
θ_{old}	Policy parameters from previous iteration (behavior policy)
$J(\theta)$	Expected return objective function
$r_t(\theta)$	Probability ratio: $\frac{\pi_{\theta_{old}}(a_t s_t)}{\pi_{\theta}(a_t s_t)}$
L_{CPI}	Conservative Policy Iteration objective
L_{CLIP}	PPO clipped surrogate objective
L^{VF}	Value function loss: $(V_{\theta}(s_t) - V_{targ}^t)^2$
$S[\pi_{\theta}]$	Entropy of policy distribution at state s
$\hat{A}_t$	Estimated advantage at time t
δ_t	TD error: $r_t + \gamma V(s_{t+1}) - V(s_t)$
γ	Discount factor
λ (GAE_LAMBDA)	GAE smoothing parameter
ϵ (CLIP_EPSILON)	Clipping interval half-width (typically 0.2)
c_1	Coefficient for value function loss
c_2	Coefficient for entropy bonus
N	Number of parallel environments
T	Number of timesteps per rollout
M	Minibatch size
K	Number of epochs (passes over data)
∇_{θ}	Gradient with respect to θ
$\hat{E}_t[\cdot]$	Empirical average over time steps
$KL(p q)$	KL divergence from distribution p to q
F	Fisher Information Matrix

References:

- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.
 - Mnih, V., Badia, A. P., Mirza, M., et al. (2016). Asynchronous Methods for Deep Reinforcement Learning. *International Conference on Machine Learning*.
-

作者信息

基本信息：杨骞玺-06年-北京理工大学-机器人工程专业；

自我介绍：一位具身领域的科研学徒，未来想长期从事具身方向的科研和创业，用机器人替代人类重复、危险的体力劳动。欢迎大家加我vx交朋友，我很丰富，无法简介；)

vx：Y1832800